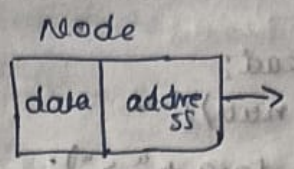
* Linked List

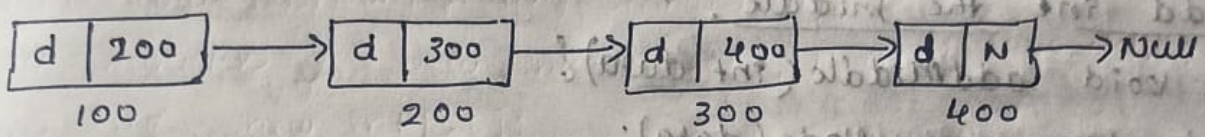
linked list is a linear data structure, which called nodes are connected using references variables in Java.

each node containes two parts

linked list do not store elements in contiguous memory location & efficient insertion & deletion.



But LL is not efficient for accessing elements by index (O(n) vs O(1) in arrays. high memory usage extra space for pointers.



reference variables will point to another object (Node).

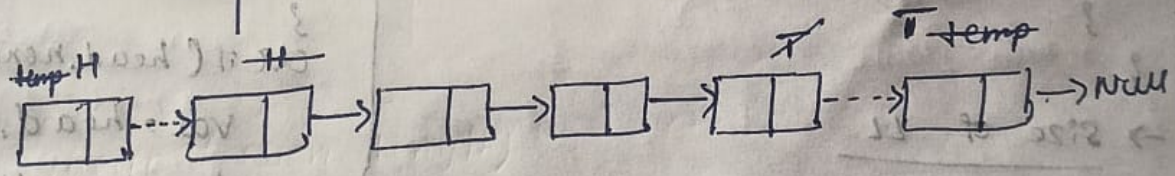
we create Node class & above nodes are objects of that class.

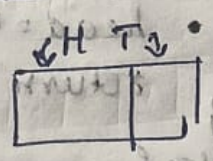
```
class Node {
    int data;
    Node next; // it is reference var so also object for that type Node.

    public Node(int d){
        data = d;
        next = null;
    }
}
```

```
public static Node head;
public static Node tail;
```

Add in LL:

* add First O(1)
* add Last O(1)
* display O(n)





Single node

```
public void addFirst(int data){
    Node temp = new Node(temp);
    if(head == null){
        head = tail = temp;
        return;
    }
    else{
        temp.next = head;
    } }  head = temp;
```

```
public void addLast(int data){
    Node temp = new Node(data);
    if(head == null){
        head = tail = null;
        return;
    }
    else{
        tail.next = temp;
        tail = temp;
    }
}
```

```
public void display(){
    if(head == null){
        S.o.p("List is empty");
        return;
    }
    Node cur = head;
    while(cur != null){
        S.o.p(cur.data + "->");
        cur = cur.next;
    }
    S.o.p("null");
}
```

```
LinkedList L = new LinkedList();
main class.
L.addFirst(4);
L.addFirst(5);
L.addLast(6);
L.addLast(7);
L.display();
L.addMiddle(22);
L.display();
```

→ Add int the middle :

```
public void addMiddle(int data){
    Node temp = new Node(data);
    if(head == null){
        head = tail = null;
    }
    else{
        S.o.p("enter ~~which~~ after which node : ");
        int till = sc.nextInt();
        Node cur = head;
        while(cur.data != till){
            cur = cur.next;
        }
        temp.next = cur.next;
        cur.next = temp;
    }
}
```

→ Size of LL

```
public int size(){
    if(head == null){
        S.o.p("empty list");
        return;
    }
    else{
        Node cur = head;
        while(cur != null){
            count++;
            cur = cur.next;
        }
    }
    return count;
}
```

int count = 0;

O(n)

O(n)

```
public int removeLast(){
    int val = 0;
    if(head == null){
        S.o.p("no list");
        return -1;
    }
    else if(head.next == null){
        val = head.data;
        head = null;
        return val;
    }
    else{
        Node cur = head;
        while(cur.next.next != null){
            cur = cur.next;
        }
        val = cur.next.data;
        cur.next = null;
        tail = cur;
        return val;
    }
}
```

O(n)

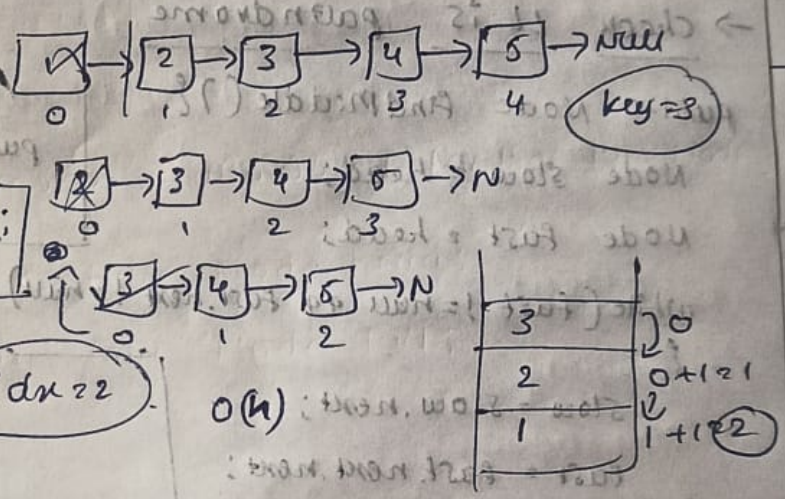
```
public int removeFirst(){
    int val=0
    if(head==null){
        S.O.P("no list");
        return -1;
    }
    else if(head.next==null){
        val = head.data;
        head = null;
        return val;
    }
    else{
        Node cur = head;
        head = cur.next;
        val = cur.data;
        cur = null;
    }
    return val;
}
```

$\boxed{O(1)}$

-> Searching element recursively

```
public int helper(Node head, int key){
    if(head == null){
        return -1;
    }
    if(head.data == key){
        return 0;
    }
    int idx = helper(head.next, key);
    if(idx == -1)
    {
        return -1;
    }
    return idx+1;
}
```

```
public int searchKey(int key){
    return helper(head,key);
}
```

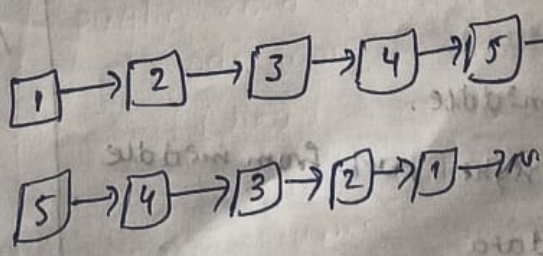


O(n)

-> Reverse linked list

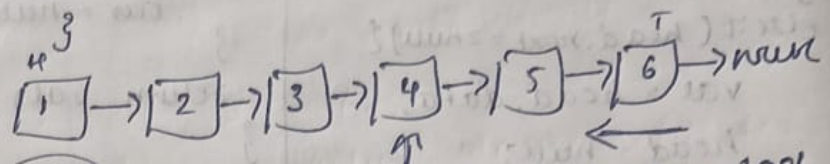
```
public void reverse(){
    Node cur = tail = head;
    Node prev = null;
    Node adrs;
    if(head == null){
        S.O.P("not possible");
    }
    else{
        while(cur != null){
            adrs = cur.next;
            cur.next = prev;
            prev = cur;
            cur = adrs;
        }
        head = prev;
    }
```

O(n)



→ delete node from the last

```
public void deleteEnd(int n){
    int sz = size();
    Node prev = head;
    if(sz==n){
        head = head.next;
        return;
    }
    else{
        int idx = 1;
        int till = sz-n;
        while(idx < till){
            prev = prev.next;
            idx++;
        }
        prev.next = prev.next.next;
        return;
    }
}
```



n=3

to be delete from last.

the node from first is = sz - n + 1.

= 6 - 3 + 1 = 4 th node from first

O(n)

→ Check LL is palindrome

```
public Node findMiddle(){
    Node slow = head;
    Node fast = head;
    while(fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

3 steps

1] finding middle.

2] reverse right part from middle

3] verify data.

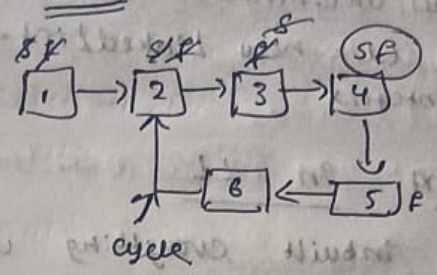
```
// actual function
public boolean palindrome(){
    Node midNode = findMiddle();
    // reverse right half.
    Node cur = midNode;
    Node prev = null;
    Node adrs;
    while(cur != null){
        adrs = cur.next;
        cur.next = prev;
        prev = cur;
        cur = adrs;
    }
    // check is it palindrome.
    Node left = head;
    Node right = prev;
    while(right != null){
        if(left.data != right.data){
            return false;
        }
        left = left.next;
        right = right.next;
    }
    return true;
}
}
```

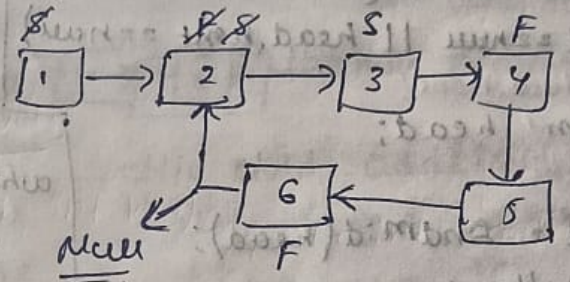
continued

→ Re detect cycle in a LL.

```
public boolean detectCycle(){
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast)
        {
            return true;
        }
    }
    return false;
}
```

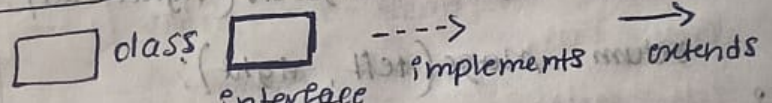
O(n)



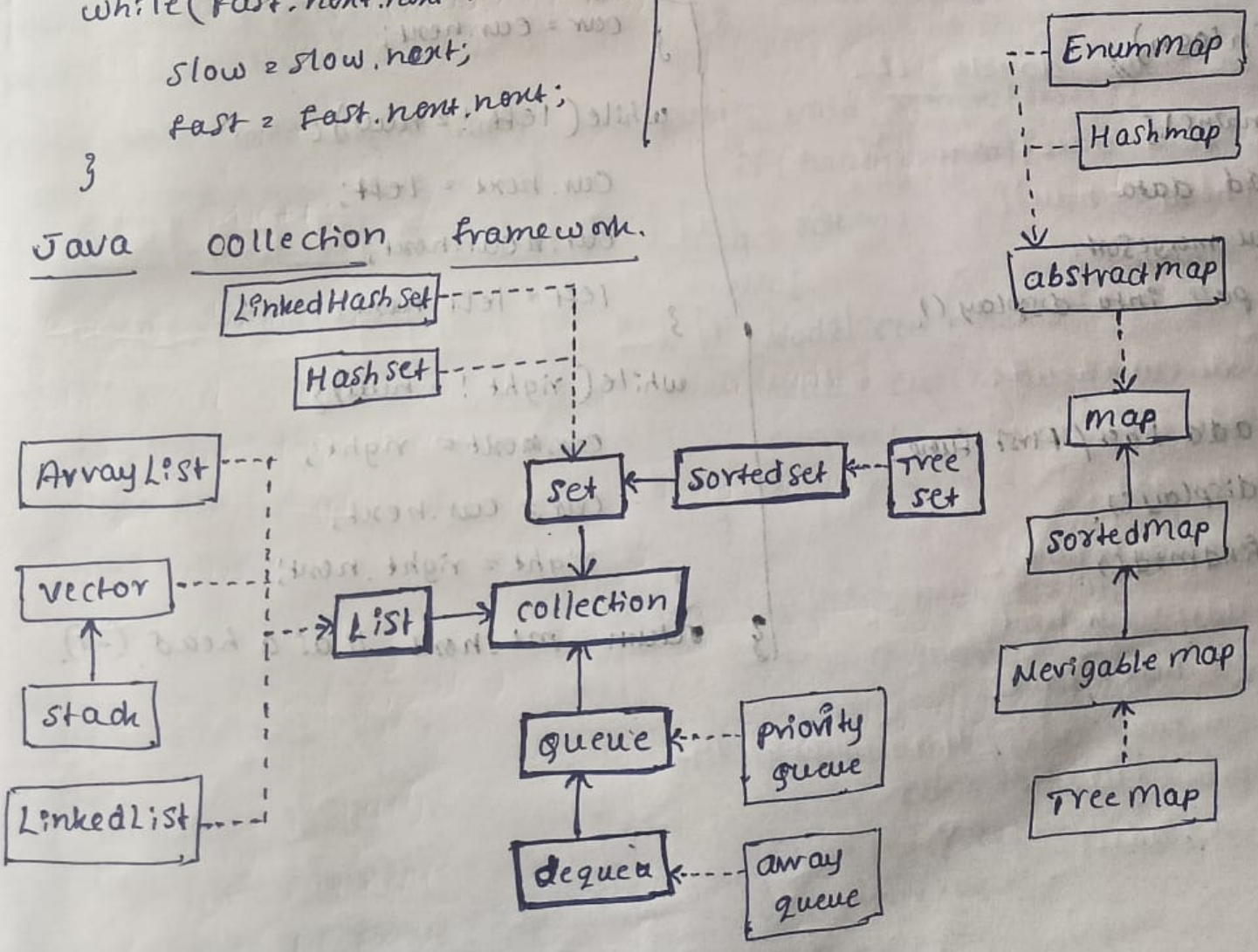


→ Remove cycle from LL.

```
public void removeCycle(){
    Node slow = head;
    Node fast = head.next;
    while(fast.next.next != slow){
        slow = slow.next;
        fast = fast.next.next;
    }
    fast.next = null;
}
```



Java collection framework.



-> LL in Java collection framework.

```
import java.util.LinkedList;
LinkedList<Integer> ll = new LinkedList<>();
```

-> merge sort in LL

don't use inbuilt everything use by writing.

```
public Node mergeSort(Node head){
    if(head==null || head.next==null)
    {
        return head;
    }
    Node mid = findMid(head);
    Node rightHead = mid.next;
    mid.next = null;
    Node left = mergeSort(head);
    Node right = mergeSort(rightHead);
    return merge(left, right);
}
```

O(nlogn)

```
in main(){
    add data
    call mergeSort.
    & pass into display()
}
```

write add Last/First func.
" display()
" findmid()

```
public Node merge(Node L, Node R)
{
    Node ML = new Node(-1);
    Node cur = ML;
    while(left != null && right != null){
        if(left.data <= right.data){
            cur.next = left;
            left = left.next;
        }
        else {
            cur.next = right;
            right = right.next;
        }
        cur = cur.next;
    }
    while(left != null){
        cur.next = left;
        cur = cur.next;
        left = left.next;
    }
    while(right != null){
        cur.next = right;
        cur = cur.next;
        right = right.next;
    }
    return ML.next // as a head, (-1).
```

* doubly linked list

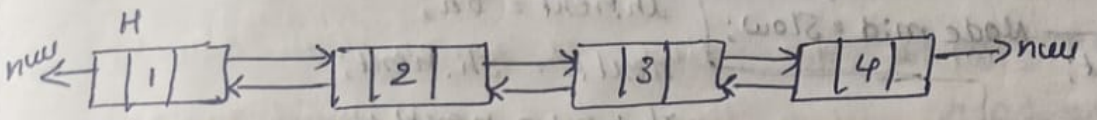
→ you can move both forward & backward which is impossible in singly.

→ it takes O(1) in insertion & deletion. No need of reverse.

→ applications like undo/redo, music playlist..

DLL in collection framework

~~Doubly dll = new~~ DoubleLL dll = new DoubleLL();




```
public class Node{
    int data;
    Node next; <- Node prev;
    Node(int d){
        data = d;
        next = null;
        prev = null;
    }
}
public static Node head;
public static Node tail;
public void addLast(int data){
    Node temp = new Node(data);
    if(head == null){
        head = ~~tail~~ = temp;
    }else{
        tail.next = temp;
        temp.prev = tail;
        tail = prev;
    }
}
public void display(){
    if(head == null) return;
    else{
        Node cur = head;
        while(cur != null){
            S.o.p(cur.data + "->");
            cur = cur.next;
        }
    }
}
```

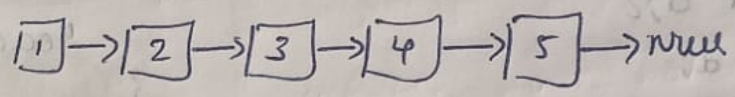
```
public void addFirst(int data){
    Node temp = new Node(data);
    if(head == null){
        head = tail = temp;
    }
    else{
        temp.next = head;
        head.prev = temp;
        head = temp;
    }
}
public void removeFirst(){
    if(head == null){
        return;
    }
    else{
        Node cur = head;
        head = cur.next;
        cur.next = null;
        head.prev = null;
    }
}
public void removeLast(){
    if(head == null){
        return;
    }else{
        Node cur = tail;
        tail = cur.prev;
        cur.prev = null;
        tail.next = null;
    }
}
```

→ Singly linked list zigzag problem:

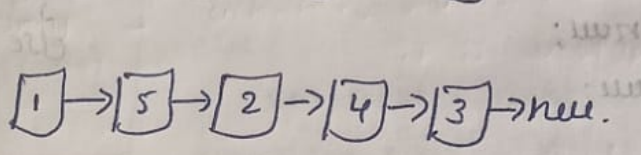
```
public void zigzag(){
  Node slow = head;
  Node fast = head;
  while( fast != null && fast.next != null)
  {
      slow = slow.next;
      fast = fast.next.next;
  }
  Node cur = slow.next;
  mid.next = null;
  Node prev = null;
  Node adrs;
  while(cur != null){
    adrs = cur.next;
    cur.next = prev;
    prev = cur;
    cur = adrs;
  }
```

← Node mid = slow;

```
  Node left = head; // lh
  Node right = prev; // rh
  while
  Node nextL, nextR;
  while(lh != null && rh != null){
      nextL = lh.next;
      lh.next = rh;
      nextR = rh.next;
      rh.next = nextL;
      lh = nextL;
      rh = nextR;
  }
```

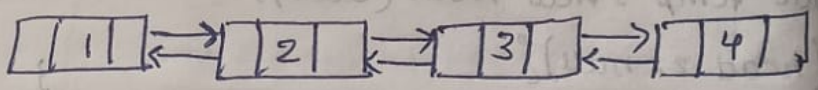


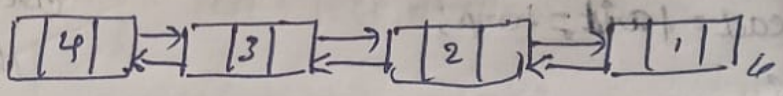
o(n)

o/p 

→ reverse the doubly LL.

```
public void doubReverse(){
    if(head == null){
        return;
    }
    else{
      Node cur = head;
      Node prev = null;
      Node adrs;
      while(cur != null){
        adrs = cur.next;
        cur.next = prev;
        cur.prev = adrs; // additional line
        prev = cur;
        cur = adrs;
      }
}
```

i/p 

o/p 

* circular LL

-> no NULL at the end, last node points to the head forms a cycle.

-> applications: Round Robin scheduling, music (repeat mode), Ring topology.

-> easy to perform all operations.

```
public void addFirst(int data){
  Node temp = new Node(data);
  if(head == null){
    head = tail = temp;
    head.next = head;
  }
  else{
    temp.next = head;
    head = temp;
    tail.next = head;
  }
}
```

```
public void addLast(int data){
  Node temp = new Node(data);
  if(head == null){
    head.tail = temp;
    head.next = head;
  }
  else{
    tail.next = temp;
    temp.next = head;
    head = temp;
  }
}
```

```
public void removeFirst(){
  if(head == null){
    return;
  }
  else{
    Node cur = head;
    head = cur.next;
    cur.next = null;
    tail.next = head;
  }
}
```

```
public void removeLast(){
  if(head == null){
    return;
  }
  else{
    Node cur = head;
    Node prev = null;
    while(cur.next != head){
      cur = cur.next;
    }
    prev.next = head;
    tail = prev;
  }
}
```

it can be swap

inserting & deleting in between is same like singly LL.